Official Documentation

Version 1.4.0 - LITE Edition

Professional Tag-Based Text Animation for Unity

Created by Spanky Boy
FluxentInteractive

Support: fluxentinteractive@gmail.com

Contents

1 Welcome to Procedural Text Animator LITE **2**
- 1.1 What is Procedural Text Animator? ... 2
- 1.2 LITE vs FULL Version ... 2
- 1.3 Key Features ... 2
- 1.4 System Requirements ... 3

2 Quick Start Guide **4**
- 2.1 Step 1: Add the Core Engine ... 4
- 2.2 Step 2: Create an Effect Database ... 4
- 2.3 Step 3: Create Effect Assets ... 4
- 2.4 Step 4: Add Effect to Database ... 4
- 2.5 Step 5: Assign Database to Engine ... 5
- 2.6 Step 6: Use Tags in Your Text! ... 5

3 Core Concepts **6**
- 3.1 The Animation Engine (PUAP_TMP) ... 6
- 3.2 The Effect Database ... 6
- 3.3 Effect Assets ... 6
- 3.4 Effect Types ... 6

4 Effect Reference **7**
- 4.1 Behavior Effects ... 7
  - 4.1.1 Wave Effect ... 7
  - 4.1.2 Shake Effect ... 7
  - 4.1.3 Pulse Effect ... 7
- 4.2 Appearance Effects ... 8
  - 4.2.1 Fade In Effect ... 8
- 4.3 Combining Effects ... 8

5 Typewriter System **9**
- 5.1 UITextTypewriter Component ... 9
- 5.2 Action Tags ... 9
  - 5.2.1 Wait Tag ... 9
- 5.3 Combining Typewriter with Effects ... 9
- 5.4 Public Methods ... 9

6 Creating Effect Variations **11**
- 6.1 Example: Multiple Wave Styles ... 11
- 6.2 Recommended Preset Library ... 11

7 Advanced Usage **12**
- 7.1 Programmatic Text Setting ... 12
- 7.2 Manual Animation Control ... 12
- 7.3 Accessing Character Data ... 12

7.4 Text Bounds ... 12
7.5 Performance Optimization ... 13

8 Creating Custom Effects (Advanced) 14
8.1 Example: Custom Bounce Effect ... 14
8.2 Understanding Character Data ... 14
8.3 Effect Creation Tips ... 15

9 Troubleshooting 16
9.1 Common Issues ... 16
9.1.1 Tags Appear as Plain Text ... 16
9.1.2 No Animation Visible ... 16
9.1.3 Typewriter Doesn't Work ... 16
9.1.4 Shake Effect Doesn't Work ... 16
9.2 Performance Issues ... 16
9.3 Getting Help ... 17

10 Best Practices 18
10.1 Project Organization ... 18
10.2 Naming Conventions ... 18
10.3 Effect Design Guidelines ... 18
10.4 Workflow Tips ... 18

11 Upgrading to FULL Version 19
11.1 What You Get in FULL ... 19
11.1.1 14 Additional Effects ... 19
11.1.2 Advanced Features ... 19
11.2 Feature Comparison ... 20
11.3 Upgrade Process ... 20
11.4 Get the FULL Version ... 20

12 Frequently Asked Questions 21
12.1 Licensing & Usage ... 21
12.2 Technical Questions ... 21
12.3 Feature Questions ... 21
12.4 Upgrade Questions ... 21

13 Quick Reference 22
13.1 Tag Syntax Reference ... 22
13.2 Component Reference ... 22
13.3 Namespace ... 22
13.4 Key Shortcuts (Unity Editor) ... 22

14 Version History 23
14.1 v1.0.0 - LITE Initial Release ... 23

15 Contact & Support 23
15.1 Support Channels ... 23

15.2 Reporting Bugs 23
15.3 Feature Requests 24
15.4 Follow Us 24

16 Thank You! 24

1 Welcome to Procedural Text Animator LITE

Thank you for choosing **Procedural Text Animator LITE**! This free version provides you with a powerful, production-ready text animation system that brings your UI to life with minimal effort.

1.1 What is Procedural Text Animator?

Procedural Text Animator is a tag-based animation engine for Unity that allows you to animate TextMeshPro text with simple tags like <wave>, <shake>, and <pulse>. No complex component setup, no animation timeline editing—just type and animate!

1.2 LITE vs FULL Version

The LITE version includes everything you need to get started:

- **Full Animation Engine** - Complete tag-based system
- **4 Professional Effects** - Wave, Shake, Pulse, and Fade
- **Typewriter System** - Character-by-character reveals with pauses
- **Commercial License** - Free for personal and commercial use
- **Production Ready** - Ship real projects with it

Full Version Feature

The **FULL version** adds 14 additional effects including Breathe, Bounce, Wobble, Glitch, Rainbow, Gradients, and more, plus advanced typewriter controls with the <speed> tag, UI Panel Shake component, and priority email support.
Upgrade at: Procedural Text Animator Pro

1.3 Key Features

Tag-Based Workflow
Simply wrap text in tags: <wave>Hello!</wave>

ScriptableObject Effects
Create reusable, tweakable effect presets

Effect Database
Centrally manage all effects for your project

High Performance
Optimized mesh manipulation with minimal GC

Extensible
Create custom effects by inheriting base classes

Cross-Platform
Works on all Unity platforms

Pipeline Compatible
Supports Built-in, URP, and HDRP

1.4 System Requirements

- Unity 2020.3 or higher
- TextMeshPro (included with Unity)
- Any render pipeline (Built-in, URP, HDRP)

2 Quick Start Guide

Get your first animation working in under 5 minutes!

2.1 Step 1: Add the Core Engine

1. Select a GameObject with a TextMeshProUGUI component
2. Click **Add Component**
3. Search for **"PUAP TMP"** and add it

Pro Tip

PUAP_TMP is the central animation engine. You only need one per text object!

2.2 Step 2: Create an Effect Database

The Effect Database is a central library that holds all available effects for your project.

1. Right-click in your Project window
2. Navigate to **Create > Procedural UI Effects Lite > Effect Database**
3. Name it something like "MyEffects"

2.3 Step 3: Create Effect Assets

Let's create a Wave effect as an example:

1. Right-click in Project window
2. Go to **Create > Procedural UI Effects Lite > Behaviors > Wave Effect**
3. Name it "MyWave"
4. In the Inspector, set the **Effect Tag** to wave
5. Adjust **Wave Height** and **Wave Speed** to taste

Info

The **Effect Tag** is the word you'll use in your text tags: <wave>...</wave>

2.4 Step 4: Add Effect to Database

1. Select your "MyEffects" database asset
2. In the Inspector, expand the **Effects** list
3. Increase the size to 1
4. Drag your "MyWave" effect into the empty slot

2.5 Step 5: Assign Database to Engine

1. Select your text GameObject
2. Find the **PUAP_TMP** component
3. Drag your "MyEffects" database into the **Effect Database** slot

2.6 Step 6: Use Tags in Your Text!

1. Select your text GameObject
2. In the TextMeshProUGUI component, set the text to:

   This is normal text, but <wave>this part waves!</wave>
3. Press Play and watch the magic!

Pro Tip

You can combine effects: <wave><pulse>Double animated!</pulse></wave>

3 Core Concepts

Understanding these core concepts will help you master the system.

3.1 The Animation Engine (PUAP_TMP)

PUAP_TMP is the heart of the system. It:

- Reads your text and parses tags
- Manages the animation state
- Modifies TextMeshPro vertex data every frame
- Applies effects from the Effect Database

Key Properties:

Effect Database The database asset containing your effects

Animation Loop When to update (Update, LateUpdate, or Manual)

Important

If no Effect Database is assigned, your tags will appear as plain text!

3.2 The Effect Database

The **Effect Database** is a ScriptableObject that acts as a central library:

- Holds all effect assets you want available
- Maps effect tags to effect implementations
- Typically one database per project
- Can be shared across scenes

When PUAP_TMP encounters a tag like <wave>, it queries the database for an effect with that tag.

3.3 Effect Assets

Effects are now **ScriptableObject assets**, which means:

- Reusable across multiple text objects
- Easy to tweak without code changes
- Can have multiple variations (e.g., ”SlowWave” and ”FastWave”)
- No component bloat on GameObjects

3.4 Effect Types

Behavior Effects
Looping animations that continuously animate text (Wave, Shake, Pulse)

Appearance Effects
One-shot animations used with the typewriter (Fade In)

4 Effect Reference

Complete reference for all effects included in LITE.

4.1 Behavior Effects

Behavior effects loop continuously and can be used anywhere in your text.

4.1.1 Wave Effect

Creates smooth, flowing sine wave motion.

Tag: <wave>...</wave>

Parameters:

- **Wave Height** - Vertical amplitude (default: 5)
- **Wave Speed** - Animation speed (default: 10)
- **Wave Frequency** - Wave density (default: 1)

Example:

```
This is <wave>wavy text!</wave>
```

Use Cases: Magical text, water themes, attention-grabbing headers

4.1.2 Shake Effect

Smooth, organic shake using Perlin noise.

Tag: <shake>...</shake>

Parameters:

- **Shake Amount** - Maximum offset distance (default: 2)
- **Shake Speed** - Shake frequency (default: 15)

Example:

```
<shake>WARNING!</shake> System critical!
```

Use Cases: Emphasis, damage indicators, alert messages, unstable text

4.1.3 Pulse Effect

Gentle alpha pulsing that creates a breathing effect.

Tag: <pulse>...</pulse>

Parameters:

- **Min Alpha** - Minimum opacity 0-1 (default: 0.3)
- **Pulse Speed** - Pulsing rate (default: 3)
- **Wave Frequency** - Character wave offset (default: 0.5)

Example:

```
Press <pulse>SPACE</pulse> to continue
```

Use Cases: UI feedback, button prompts, breathing life into static text

4.2 Appearance Effects

Appearance effects are one-shot animations used with the UITextTypewriter.

4.2.1 Fade In Effect

Smoothly fades characters from transparent to opaque.

Tag: <fade>...</fade> (used with typewriter)

Parameters:

- **Duration** - Fade duration in seconds (default: 0.5)

Example:

```
// Attach UITextTypewriter component
<fade>Hello, welcome to my game!</fade>
```

Use Cases: Professional typewriter reveals, dialogue systems

Full Version Feature

The FULL version includes 3 additional appearance effects:

- **Drop In** - Characters fall from above
- **Scale Pop** - Bounce reveal with scaling
- **Rotate Flip** - 3D rotation reveal

4.3 Combining Effects

You can stack multiple effects on the same text:

```
<wave><pulse>Maximum impact!</pulse></wave>
```

Effects are applied in the order they're nested.

5 Typewriter System

The typewriter reveals text character-by-character with smooth timing.

5.1 UITextTypewriter Component

Add this component to the same GameObject as PUAP_TMP to enable typewriter function- ality.

Key Properties:

Chars Per Second Base typing speed (default: 20)

Play On Enable Auto-start when enabled (default: true)

On Typing Complete UnityEvent fired when finished

5.2 Action Tags

Action tags control typewriter behavior during typing.

5.2.1 Wait Tag

Pauses the typewriter for a specified duration.

Syntax: <wait=X> where X is seconds

Example:

```
Hello...<wait=1>My name is...<wait=0.5>Claude!
```

The typewriter will pause for 1 second after "Hello...", then 0.5 seconds after "My name is...".

Full Version Feature

The FULL version includes the <speed> tag:
Normal <speed=2>faster</speed> <speed=0.5>slower</speed>
This dynamically changes typing speed during playback!

5.3 Combining Typewriter with Effects

You can use behavior effects within typewriter text:

```
Hello, I am a <shake>glitchy</shake> AI!<wait=1> Nice to meet you.
```

The word "glitchy" will shake while being typed out.

5.4 Public Methods

Control the typewriter programmatically:

```
public void StartTyping()
public void StopTyping()
public void CompleteInstantly()
```

Example Usage:

```
using ProceduralUIEffects.Lite;

public class DialogueManager : MonoBehaviour
{
    public UITextTypewriter typewriter;

    void Update()
    {
        // Skip to end when space pressed
        if (Input.GetKeyDown(KeyCode.Space))
        {
            typewriter.CompleteInstantly();
        }
    }
}
```

6 Creating Effect Variations

One of the most powerful features is creating multiple variations of effects without writing code.

6.1 Example: Multiple Wave Styles

Let's create three wave variations:

1. **Create three Wave Effect assets:**
   - Wave_Gentle
   - Wave_Normal
   - Wave_Energetic
2. **Configure each with different settings:**

Asset	Tag	Height	Speed
Wave_Gentle	gentle	3	5
Wave_Normal	wave	5	10
Wave_Energetic	energy	10	20

3. **Add all three to your Effect Database**
4. **Use in your text:**

```
This is <gentle>gentle</gentle>, this is <wave>normal</wave>,
and this is <energy>ENERGETIC!</energy>
```

6.2 Recommended Preset Library

For a professional demo or production game, create these presets:

Wave Variations:

- Wave_Gentle (tag: gentle)
- Wave_Normal (tag: wave)
- Wave_Energetic (tag: energy)

Shake Variations:

- Shake_Subtle (tag: shake)
- Shake_Dramatic (tag: dramatic)

Pulse Variations:

- Pulse_Slow (tag: pulse)
- Pulse_Fast (tag: pulsefast)

This gives you 7 ready-to-use effect variations!

7 Advanced Usage

7.1 Programmatic Text Setting

Set text dynamically from code:

```
using ProceduralUIEffects.Lite;

public class DynamicText : MonoBehaviour
{
    public PUAP_TMP animator;

    void Start()
    {
        animator.SetText("Hello <wave >dynamic </wave > world!");
    }
}
```

7.2 Manual Animation Control

For advanced control, use Manual animation mode:

```
public PUAP_TMP animator;

void Awake()
{
    animator.animationLoop = AnimationLoop.Manual;
}

void Update()
{
    // Call with custom delta time
    animator.Animate(Time.deltaTime * customSpeed);
}
```

7.3 Accessing Character Data

Access individual character data for custom logic:

```
for (int i = 0; i < animator.CharactersCount; i++)
{
    CharacterData character = animator.characters[i];

    // Check character position
    Vector3 position = character.sourceVertices[0];

    // Check time character has been visible
    float timePassed = character.passedTime;
}
```

7.4 Text Bounds

Get the bounds of visible text:

```
Vector3 minBounds = animator.textMinBounds;
Vector3 maxBounds = animator.textMaxBounds;
```

```
// Calculate text size
Vector3 textSize = maxBounds - minBounds;
```

7.5 Performance Optimization

Best Practices:

- Use one Effect Database per project (reuse across scenes)
- Limit effects to visible text only
- Use AnimationLoop.Manual for precise control
- Pool and reuse PUAP_TMP GameObjects when possible

8 Creating Custom Effects (Advanced)

Advanced users can create custom effects by inheriting from BehaviorEffect or AppearanceEffect.

8.1 Example: Custom Bounce Effect

```
using UnityEngine;
using ProceduralUIEffects.Lite;

[CreateAssetMenu(menuName = "My Custom Effects/Bounce")]
public class CustomBounceEffect : BehaviorEffect
{
    [Header("Bounce Settings")]
    public float bounceHeight = 10f;
    public float bounceSpeed = 5f;

    public override void ApplyEffect(ref CharacterData character,
                                     PUAP_Core animator)
    {
        // Calculate bounce offset
        float time = animator.timeSinceStart * bounceSpeed;
        float yOffset = Mathf.Abs(Mathf.Sin(time)) * bounceHeight;

        // Apply to all 4 vertices
        for (int i = 0; i < 4; i++)
        {
            character.modifiedVertices[i] +=
                new Vector3(0, yOffset, 0);
        }
    }
}
```

8.2 Understanding Character Data

Each character has 4 vertices (corners) that you can modify:

- character.sourceVertices[] - Original positions
- character.modifiedVertices[] - Modified positions (what you change)
- character.sourceColors[] - Original colors
- character.modifiedColors[] - Modified colors
- character.index - Character index in text
- character.passedTime - Time since character was revealed

8.3 Effect Creation Tips

Pro Tip

- Always work with modifiedVertices/Colors, not source data
- Use animator.timeSinceStart for time-based effects
- Use character.index for per-character variation
- Test with different text lengths and fonts

9 Troubleshooting

9.1 Common Issues

9.1.1 Tags Appear as Plain Text

Problem: Tags like <wave> show up in text instead of animating.

Solutions:

- Verify Effect Database is assigned to PUAP_TMP
- Check that effect tag matches exactly (case-sensitive!)
- Ensure effect is added to the database's Effects list

9.1.2 No Animation Visible

Problem: Text appears but doesn't animate.

Solutions:

- Check Animation Loop is set to Update or LateUpdate
- Verify the effect asset has non-zero values
- Ensure the GameObject is active and enabled
- Check Console for any error messages

9.1.3 Typewriter Doesn't Work

Problem: UITextTypewriter component doesn't reveal text.

Solutions:

- Ensure UITextTypewriter is on the same GameObject as PUAP_TMP
- Check "Play On Enable" is enabled
- Verify "Chars Per Second" is greater than 0
- Make sure you're using an appearance effect tag like <fade>

9.1.4 Shake Effect Doesn't Work Problem:

Shake effect shows no movement.

Solution:

- This is normal! Shake uses characterSeeds from PUAP_TMP
- Make sure you're using the LITE version of all scripts
- Verify the PUAP_TMP.cs file includes the characterSeeds property

9.2 Performance Issues

If animations feel sluggish:

1. Reduce the number of animated characters
2. Use simpler effects (Wave instead of complex custom effects)
3. Check your target framerate settings

4. Profile your game to identify bottlenecks

9.3 Getting Help

If you're still stuck:

- Check the included demo scene for working examples
- Review the Quick Start Guide (Chapter 2)
- Email support: fluxentinteractive@gmail.com
- Visit the Unity Forum thread for community help

10 Best Practices

10.1 Project Organization

Recommended folder structure:

```
Assets/
    TextAnimator/
        Effect Database.asset
        Effects/
            Behaviors/
                Wave_Gentle.asset
                Wave_Normal.asset
                Shake_Subtle.asset
                Pulse_Slow.asset
            Appearances/
                Fade_Quick.asset
```

10.2 Naming Conventions

- **Database:** EffectDatabase or TextEffects
- **Effects:** EffectType_Variation (e.g., Wave_Gentle)
- **Tags:** Lowercase, short, memorable (e.g., wave, shake)

10.3 Effect Design Guidelines

Subtle is Better
Start with subtle animations, increase intensity if needed

Performance First
Test effects with long text passages

Readability
Ensure animated text is still readable

Context Appropriate
Match effect intensity to context (gentle for body text, dramatic for titles)

10.4 Workflow Tips

1. Create one Effect Database per project
2. Build a preset library early (7-10 variations)
3. Name tags consistently across your project
4. Test effects with different fonts and sizes
5. Keep a demo scene with all effects visible

11 Upgrading to FULL Version

The FULL version of Procedural Text Animator significantly expands your creative toolkit!

11.1 What You Get in FULL

11.1.1 14 Additional Effects

Behavior Effects (11 more): 2

- Breathe - Scale pulsing
- Bounce - Bouncing motion
- Wobble - Jelly-like wobble
- Swing - Pendulum swing
- Shear - Slanting motion
- Jitter - Erratic jumps
- Glitch - RGB chromatic split
- Rainbow - Color cycling
- Gradient - Static gradients
- Animated Gradient - Scrolling colors

Appearance Effects (3 more):

- Drop In - Fall from above reveal
- Scale Pop - Bounce reveal with overshoot
- Rotate Flip - 3D rotation reveal

11.1.2 Advanced Features

Speed Tag
Dynamically change typing speed: <speed=2>faster!</speed>

UI Panel Shake
Shake entire UI panels and RectTransforms

Priority Support
Email support with 24-hour response time

Professional Documentation
Complete PDF manual with advanced examples

Demo Scenes
Full showcase with all 18 effects

11.2 Feature Comparison

Feature	LITE	FULL
Core Engine Tag		
System Effect		
Database		
Behavior Effects	3	14
Appearance Effects	1	4
Typewriter (wait tag)		
Typewriter (speed tag)		
Panel Shake Component		
Custom Effect Creation		
Commercial License		
Email Support	Community	Priority

11.3 Upgrade Process

1. Purchase FULL version from Unity Asset Store
2. Both versions can coexist (different namespaces)
3. Import FULL version into your project
4. Gradually migrate effects to FULL namespace as needed

Info

LITE and FULL versions use different namespaces (.Lite vs default), so they won't conflict!

11.4 Get the FULL Version

Visit Unity Asset Store

PROCEDURAL TEXT ANIMATOR PRO

Unlock the complete creative toolkit!

12 Frequently Asked Questions

12.1 Licensing & Usage

Q: Can I use LITE in commercial projects?
A: Yes! The LITE version is completely free for both personal and commercial use.

Q: Do I need to credit you?
A: Not required, but always appreciated! You can credit "FluxentInteractive" if you'd like.

Q: Can I modify the source code?
A: Yes, full C# source code is included and you're free to modify it for your projects.

12.2 Technical Questions

Q: Does this work with Unity's built-in Text component?
A: No, this asset requires TextMeshPro (which comes free with Unity).

Q: Will LITE and FULL conflict if I import both?
A: No! They use different namespaces and can coexist peacefully.

Q: Can I create my own custom effects?
A: Yes! See Chapter 8 for a guide on creating custom effects.

Q: What platforms are supported?
A: All Unity platforms - PC, Mac, Linux, Mobile, WebGL, Consoles, etc.

12.3 Feature Questions

Q: How do I get the speed tag for typewriter?
A: The <speed> tag is a FULL version feature. LITE includes <wait> only.

Q: Can I stack multiple effects on the same text?
A: Yes! Use nested tags: <wave><pulse>text</pulse></wave>

Q: How many effects can I have in one scene?
A: There's no hard limit. Performance depends on total character count being animated.

12.4 Upgrade Questions

Q: If I upgrade, will my LITE work still work?
A: Yes! Both versions can coexist. You can migrate at your own pace.

Q: Is the upgrade worth it?
A: If you need more variety or advanced features (Glitch, Rainbow, speed tags), absolutely! Many users upgrade after trying LITE.

13 Quick Reference

13.1 Tag Syntax Reference

Tag	Type	Example
<wave>	Behavior	<wave>text</wave>
<shake>	Behavior	<shake>text</shake>
<pulse>	Behavior	<pulse>text</pulse>
<fade>	Appearance	<fade>text</fade>
<wait=X>	Action	<wait=1.5>

13.2 Component Reference

PUAP_TMP

Core animation engine - attach to TextMeshProUGUI

UITextTypewriter

Character-by-character reveal - attach to same GameObject as PUAP_TMP

EffectDatabase

ScriptableObject holding all effects - assign to PUAP_TMP

13.3 Namespace

All LITE classes are in the ProceduralUIEffects.Lite namespace:

```
using ProceduralUIEffects.Lite;
```

13.4 Key Shortcuts (Unity Editor)

Action	Path
Create Database	Create > Procedural UI Effects Lite > Effect Database Create
Wave	Create > Procedural UI Effects Lite > Behaviors > Wave Create
Shake	Create > Procedural UI Effects Lite > Behaviors > Shake Create
Pulse	Create > Procedural UI Effects Lite > Behaviors > Pulse Create Fade
	Create > Procedural UI Effects Lite > Appearances > FadeIn
Upgrade Menu	Procedural UI Effects Lite > Upgrade to Full Version

14 Version History

14.1 v1.0.0 - LITE Initial Release

Release Date: [Your Date]

Features:

- Initial release of LITE version
- Full tag-based animation engine
- 4 core effects: Wave, Shake, Pulse, Fade
- UITextTypewriter with wait tag
- Effect Database system
- Commercial license included
- Complete documentation

Known Limitations:

- No speed tag (Full version feature)
- Limited to 4 effects (Full has 18)
- No Panel Shake component (Full version feature)

15 Contact & Support

15.1 Support Channels

Email
fluxentinteractive@gmail.com

Response Time
Community support (48-72 hours)

Unity Forum
[Link to your forum thread]

GitHub Issues
[Link if you have one]

Info

Full Version users get priority email support with 24-hour response time!

15.2 Reporting Bugs

When reporting bugs, please include:

1. Unity version
2. LITE version number

3. Render pipeline (Built-in/URP/HDRP)

4. Steps to reproduce
5. Console errors (if any)
6. Screenshot or video if possible

15.3 Feature Requests

We love hearing your ideas! Send feature requests to our email. Note that new effects will typically be added to the Full version first.

15.4 Follow Us

- Twitter: @FluxentInt (if applicable)
- Discord: [Your Discord] (if applicable)
- Website: [Your Website] (if applicable)

16 Thank You!

Thank you for using **Procedural Text Animator LITE**! We hope this tool helps you create amazing text animations in your Unity projects.

If you find LITE useful, please consider:

- Rating it on the Unity Asset Store
- Upgrading to the Full version
- Sharing it with fellow developers
- Leaving a review with feedback

Happy Animating!

Made with by Spanky Boy
FluxentInteractive

"Bring your text to life!"